

UART_ESP32 - Projektdokumentation

Firmware Datei-Upload - v0.25

Robuster Browser -> ESP32 -> BX3 Datei-Upload mit segmentierter UART-Uebertragung und erweiterter Diagnose

Projektname	UART_ESP32
Dokumentenart	Projektdokumentation / Firmware-Bericht
Thema	Datei-Upload - UART-Segmentierung und Diagnose
Erstellungsdatum	2026-09-22
Aenderungsdatum	2026-09-22
Verantwortlich	Projekt UART_ESP32 - technische Erstellung: ChatGPT
Version	v0.25
Status	In Pruefung - Source-Aenderung erstellt, PlatformIO-/Hardwaretest offen
Ablageort	Uart_ESP32/docs/
Referenzen	Firmware v0.24, VALIDATION_v0.25.txt, Commit-Nachricht v0.25, Projektdokumentationsregeln vom 2026-06-18

1. Anlass und Ziel

In Firmware v0.24 konnte der neue Datei-Upload beim ersten 2048-Byte-Block mit "UART Timeout" abbrechen. Der Browser hatte den Block bereits an den ESP32 uebergeben, die BX3-Shell lieferte jedoch nicht den erwarteten Transfermarker zurueck. Ziel von v0.25 ist, den identifizierten langen UART-Shellbefehl zu vermeiden und gleichzeitig den Ablauf so zu dokumentieren, dass ein Fehler im Webinterface einer konkreten Transferphase zugeordnet werden kann.

2. Technische Ursache in v0.24

v0.24 kodierte einen kompletten 2048-Byte-Block in Base64 und uebertrug die Daten gemeinsam mit Shell-Logik als eine einzelne UART-Zeile. 2048 Byte ergeben bereits etwa 2732 Base64-Zeichen. Zusammen mit Dateipfad, base64 -d, wc -c, cksum und Transfermarkern entstand eine mehrere Kilobyte lange Eingabezeile. Wenn die BX3-Shell bzw. ein vorgeschalteter TTY-/UART-Puffer diese Zeile nicht vollstaendig verarbeitete, blieb die erwartete CHUNK-Bestaetigung aus. Nach 15 Sekunden UART-Stille meldete der ESP32 den Timeout.

3. Loesung in v0.25

- Der Browser arbeitet weiterhin mit logischen 2048-Byte-Blocken.
- Upload-Zielpfade sind auf 80 Zeichen begrenzt, damit auch bei maximaler Pfadlaenge kurze UART-Shellzeilen erhalten bleiben.
- Der ESP32 zerlegt jeden logischen Block fuer die UART-Shell in 96-Byte-Teilsegmente.
- Jedes Teilsegment wird Base64-kodiert und als kurze Shellzeile an die BX3 gesendet.
- Nach jedem Teilsegment muss die BX3 einen SEG-Marker mit Blockoffset, Segmentnummer, Segmentlaenge und aktueller temporaerer Blocklaenge bestaetigen.
- Erst nach allen Teilsegmenten wird der komplette temporaere Block mit POSIX cksum geprueft.
- Nur ein korrekt verifizierter Block wird an die .part-Datei angehaengt.
- Segment-Timeouts, Segmentfehler und Blockpruefungsfehler fordern eine Wiederholung des gesamten logischen Blocks an; maximal fuenf Retries.
- Die finale Zieldatei entsteht weiterhin erst nach Gesamtgroessen- und Gesamt-CRC-Pruefung durch mv; optional folgt chmod +x.

4. Transferablauf

Browser -> ESP32: 2048 Byte logischer Block

ESP32 -> BX3: 96-Byte-Segmente, jeweils Base64 + kurze Shellzeile

BX3 -> ESP32: SEG-Bestaetigung nach jedem Segment

BX3 Blockpruefung: wc -c + POSIX cksum

Erfolg: temporaerer Block wird an <ziel>.part angehaengt und als CHUNK bestaetigt

Abschluss: Gesamtgroesse + Gesamt-cksum, danach mv auf den Zielnamen und optional chmod +x

5. Verbesserte Fehlerdiagnose

Die Seite "UART Datei-Upload" zeigt in v0.25 zusaetzlich die aktuelle Phase, die UART-RX-Stille, das aktuelle Teilsegment, Segmentgroesse, Fehlerzaehler und Block-Retries. Dadurch kann ein Abbruch auf Vorbereitung, Segmenttransport, Blockpruefung oder Abschlusspruefung eingegrenzt werden.

Phase	Bedeutung	Typische Diagnose
prepare	Verzeichnis, .part-Datei und Werkzeuge vorbereiten	PREPARE_PATH_OR_WRITE oder TOOLS_OR_BASE64_DECODER
segment	Kurzes UART-Teilsegment senden und auf SEG warten	Segmentnummer, Offset, RX-Stille; bei Timeout Block-Retry
chunk_verify	Kompletten logischen Block mit cksum pruefen	Groesse/CRC; bei Fehler Block-Retry
verify	Gesamtdatei pruefen und umbenennen	Gesamtgroesse oder Gesamt-CRC
done	Datei erfolgreich verifiziert	Zieldatei ist freigegeben

6. Vorbereitung und Verzeichnisbehandlung

Das Zielverzeichnis wird mit mkdir -p automatisch angelegt. v0.25 prueft die Vorbereitung expliziter: Wenn Verzeichnisanlage, Loeschen alter Temporaerdateien oder das Anlegen der .part-Datei fehlschlaegt, antwortet die BX3 mit PREPARE_PATH_OR_WRITE. Zudem wird nicht nur geprueft, ob base64 und cksum existieren, sondern ob base64 -d tatsaechlich einen Testwert dekodieren kann. Ein fehlendes oder inkompatibles Werkzeug fuehrt zu TOOLS_OR_BASE64_DECODER statt zu einem unspezifischen Vorbereitungstimeout.

7. Geaenderte Dateien

- README.md
- VALIDATION_v0.25.txt
- src/UartManager.h
- src/UartManager.cpp
- src/main.cpp
- docs/2026-09-22 - Uart_Esp32 - Commit-Nachricht - Robuster-Datei-Upload-und-Diagnose - v0.25.txt
- docs/2026-09-22 - Uart_Esp32 - Projektdokumentation - Firmware - v0.25.docx
- docs/2026-09-22 - Uart_Esp32 - Projektdokumentation - Firmware - v0.25.pdf

8. Validierungsstatus

Die Source-Aenderungen wurden in dieser Umgebung auf Zustandslogik, Retry-Verhalten, JSON-Statusausgabe, temporaere Dateibehandlung und C++-Delimiter/String-Struktur geprueft. Eine PlatformIO-

ESP32-Toolchain steht in dieser Laufzeit nicht zur Verfügung. Deshalb sind der reale PlatformIO-Build, das Flashen und der End-to-End-Test mit der konkreten BX3 weiterhin als Abnahmetest erforderlich.

- Build: `pio run -e uart_esp32_ota`
- Firmwareseite muss v0.25 anzeigen.
- Zuerst kleine Testdatei nach `/tmp/bx3test/test.bin` hochladen.
- Danach `build/ptybridge` nach `/tmp/bx3test/ptybridge` hochladen.
- Bei Fehlern Status, Phase, Teilsegment, RX-Stille, Fehler und Retries dokumentieren.

9. Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.24	2026-09-22	Projekt UART_ESP32	Arbeitsstand	Erster Browser -> BX3 Datei-Upload; Buildfix fuer JSON-Status.
v0.25	2026-09-22	Projekt UART_ESP32	In Pruefung	UART-Upload segmentiert, Retry-Verhalten und Diagnose erweitert; Vorbereitung differenziert Fehler.

10. Auslieferung

Die Auslieferung erfolgt gemaess Projektdokumentationsregeln mit einem vollstaendigen Projekt-ZIP, einem ZIP nur mit geaenderten/neu benoetigten Dateien in Originalstruktur, einer Patch-Datei, einer englischen Commit-Nachricht sowie dieser Dokumentation als PDF und bearbeitbarer DOCX-Datei. Der Root-Ordner im Projekt-ZIP bleibt `Uart_ESP32`.